

Very brief SPIM tutorial for 3827

Prof. Rubenstein

March 2, 2026

1 SPIM Overview

SPIM is a MIPS32 Emulator that we will use. You can download and see detailed documentation at <http://spimsimulator.sourceforge.net>. SPIM will *emulate execution* your MIPS assembly code, which means that the code is not assembled into machine code, and is not directly run on the microprocessor of your machine (your machine's microprocessor is probably not directly MIPS-compatible anyhow).

A simple sample SPIM program, available at <http://www.cs.uic.edu/~troy/spring04/cs366/ex2.html>, is presented below to familiarize us with SPIM.

In class, we cover the MIPS instruction set architecture. To write a program, there are a few more things we need to cover:

- Data and code “placement” in memory: we need to instruct the SPIM simulator where to put our code and the code's data in (SPIM's simulated) memory. Rather than put the code at a fixed address, we can label the various data items we need placed and then let the simulator choose the addresses automatically (and in real assembly code, the assembler could similarly choose the memory locations).
- Pseudo-instructions: MIPS is a RISC (reduced instruction set) architecture, so there are a limited set of simple instructions made available. More complex operations will be represented as a pseudo-instruction. The SPIM assembler recognizes the pseudo-instructions and knows how to convert it into a (series of) actual MIPS instructions.

A simple example of a pseudo-instruction is:

```
move $a0, $s0
```

which copies the value from register \$s0 into register \$a0. This is not a “real” MIPS instruction, but the MIPS assembler can convert this instruction to:

```
add $a0, $s0, $zero
```

which takes the value in \$s0, adds the zero registers value (which is 0) and stores that result in \$a0. That's about the simplest pseudo-instruction there is. Some are more complex, discussed later.

First, look at the entire Sample SPIM program (on the next page so it's all on a single page):

```

# Sample spin program
#
# Written by Pat Troy, 11/8/2002
.data
prompt: .asciiz "Enter in an integer: "
str1: .asciiz "the answer is: "
newline: .asciiz "\n"
bye: .asciiz "Goodbye!\n"
.globl main

.text
main:

# initialize
li $s0, 10

# prompt for input
li $v0, 4
la $a0, prompt
syscall

# read in the value
li $v0, 5
syscall
move $s0, $v0

loop:
# print str1
li $v0, 4
la $a0, str1
syscall

# print loop value
li $v0, 1
move $a0, $s0
syscall

# print newline
li $v0, 4
la $a0, newline
syscall

# decrement loop value and branch if not negative
sub $s0, $s0, 1
bgez $s0, loop

# print goodbye message
li $v0, 4
la $a0, bye
syscall
li $v0, 10 ## Added by Prof. Rubenstein for proper termination
syscall ## Added by Prof. Rubenstein for proper termination

```

2 Breakdown of the code

- **Comments:** On any line of code, anything after a '#' is a comment and is ignored by the assembler (it's there to help understand the code).
- **Labels:** On any line that starts with alpha-numeric text and ends with a ':' is a label. What immediately follows the label (code or program data) will be stored in memory at some address and that label can be used within your assembly code to reference that address. The assembler will determine the address.
- **Directives:** Text preceded by a '.' are directives. They provide the assembler with some special information. In this example we have:
 - **.data:** Tells the assembler that the following information is **program data** that should be stored in memory. For instance, ASCII strings that need to be printed are often stored here, each with its own label for reference by the program.
 - **.ascii:** An ASCII string that should be stored in memory (often also provided with its own label so that it can be directly accessed by the program), and terminated with a NULL (ASCII value 0) character. Note that the **.ascii** directive doesn't provide the NULL termination. However, when ASCII text is printed, the printing stops when a NULL character is reached, so it's generally important to include the NULL character.
 - **.globl:** Followed by a label, that label is now *global*, meaning that it can be accessed by MIPS assembler code in another file (like declaring a function in a header file in c). We don't need this, and it didn't have to be in the sample code.
 - **.text:** informs the assembler that what follows is code.
- **Pseudo-instructions:** these are instructions that are not actually part of MIPS, but which can easily be converted to (one or more) MIPS instructions by the assembler:
 - **li:** Load-immediate: loads a constant into a register. `li REG const` maps directly to `ORI REG $zero const` for a constant no larger than 2^{16} . For larger constants, it maps to the combination `LUI` (load-upper-immediate) and `ORI` (to be discussed in class).
 - **la:** Load-address into a register. This pseudo-instruction is useful because a) it can be given a label for some data and the assembler will figure out the corresponding address and b) addresses are 32-bit quantities, and since an instruction needs to use some bits of the 32-bit instruction word to indicate the instruction, the 32 bits (sometimes) cannot be passed in within a single instruction. Generally, **la** will be converted to the pair of instructions **lui** (load-upper-immediate) which loads a 16-bit constant into the 16 **high-order** bits and zeros the 16 low order bits, followed by an **ori** (or-immediate) that fills in the remaining 16 bits. Note that in some cases, the address may have all 0's in either the highest or lowest 16 significant bits, in which case, the pseudo-instruction can map to a single instruction.
 - **syscall:** This is a very powerful pseudo-instruction. Its meaning is to call make a *system call* to the kernel (perhaps implemented as an interrupt or as a procedure call, the details are not spelled out). **syscall** can perform one of several tasks - the task which it is to perform is indicated by the value to which `$v0` is set, and the task is performed on the data in (or, depending on the value of `$v0`, the data at the address) stored in `$a0`. The tasks are (for the value in `$v0`):
 1. print integer
 2. print float
 3. print double
 4. print string (`$a0` holds address of string)
 5. read integer
 6. read float
 7. read double

8. read string (\$a0 holds address to write to)
 9. sbrk (allocate memory, like malloc, in bytes)
 10. exit (terminate the program)
 11. print character
 12. read character
 13. open (file)
 14. read (from file)
 15. write (to file)
 16. close
 17. exit2 (exit and return a value)
- **move** and **sub**: discussed in class. **move** is simply **add** with the \$zero register, and **sub** is **addi** with the constant negated.

2.1 Small error in code on webpage

Note that there is a small error/oversight in the code on the webpage. The main function is itself a procedure, and should **return** control when done. Without the **jr** instruction at the end, the emulator will register an error as it calls “goodbye” but then fails to return control - control just keeps moving forward and runs into garbage.

3 Using SPIM

What is described here is based on the QTSpim emulator, as run on a Mac. The main window looks like this (colored boxes added by Prof. Rubenstein):

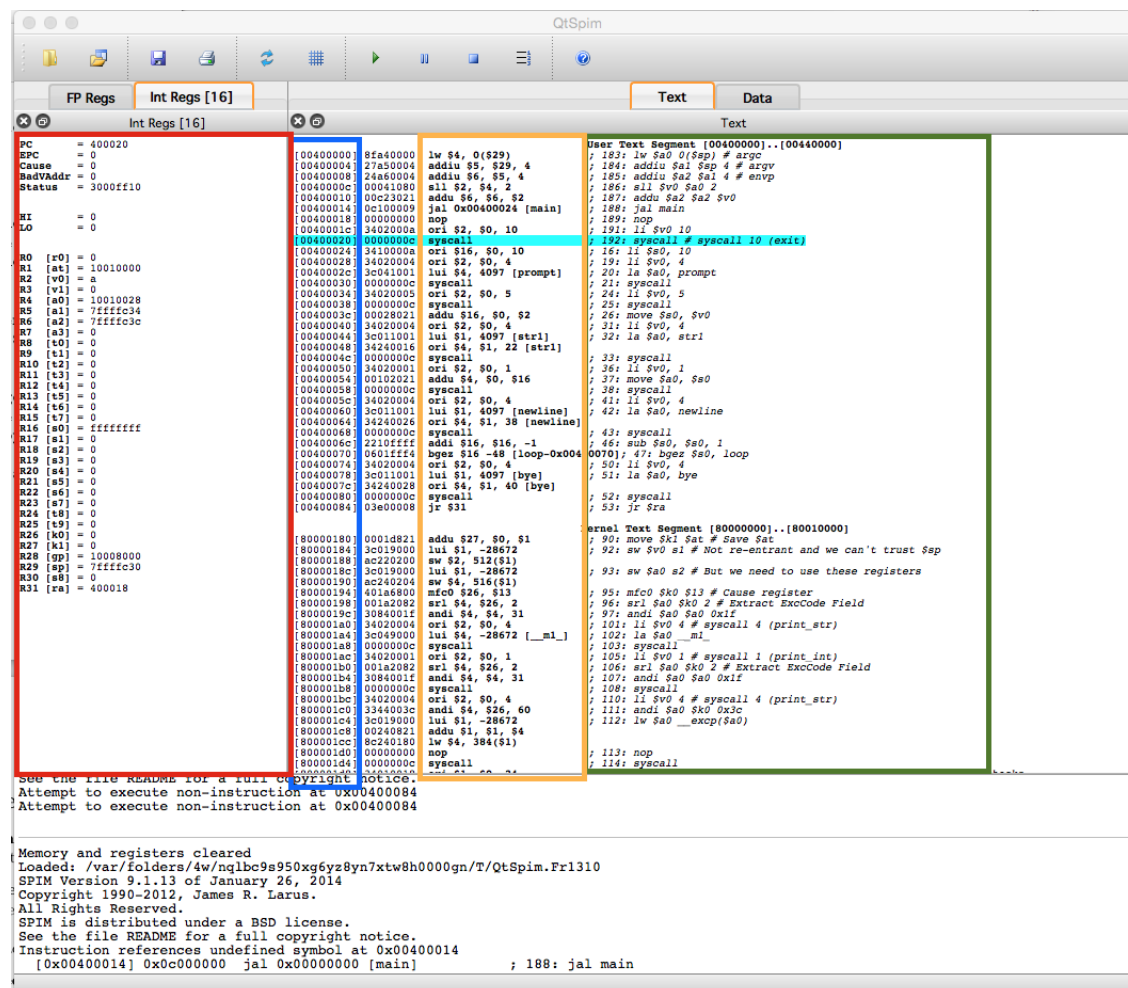


Figure 1: QTSpim as it appears on a Mac

Here is what you are looking at. This window contains several sub-windows - the views of the sub-windows can be modified.

- Red box: the register file (and some additional registers outside the register file). All 32 registers in the register file are listed, along with their values (in hex, though there is an option to view in decimal or binary). Some “special” registers are listed above the register file. Of specific importance to us is **PC**: the program counter, the register that “points” to the address containing the current instruction being executed. In the snapshot, the PC equals 400020 (hex) which means that the instruction at 400020 is currently (next) to be executed. Note that the “Int Regs” are currently showing. There are also floating point registers in SPIM - we won’t be using those.
- Blue box: Addresses. We are looking at the “Text” window, which means we are looking at the code. Note that the address held by the PC is highlighted (light blue) to indicate that this address holds the “current” instruction.
- Orange box: The “actual” instructions, any pseudo-code has been converted (note that **syscall** is still there, so it should be considered a special instruction with no substitute).

- Green box: comments, which describe the (pseudo)-code that led to the given instruction. Note, for instance, the commented line numbered 51 (la \$a0, bye). This pseudo-instruction is converted into two real instructions. Note also that the actual code has inserted values for labels (for some reason, the decimal value is shown, which is particularly annoying when it corresponds to the 16 higher-order bits of an address, e.g., line 51: 4097 (which is 1001 in hex) is loaded into the high order bits of \$a0, and the “bye” data in fact resides at (hex) address 10010028 (i.e. in decimal, $4097 \cdot 2^{16} + 40$).

Note that the combination of information in the green box (your code) and orange box (actual instructions) is helpful to see how labels are mapped to addresses, and to see the mapping of register names (e.g., \$s0) to its actual register number (e.g., \$16 for \$s0).

4 Running SPIM

This documentation is specific to the version on a Mac, although other versions are probably very similar, specific key clicks are likely to be different.

4.1 Loading and Running Code

Since your code should be in a single file, use the “File → Load and Reinitialize” option when loading in your code.

You can run your code by choosing “Simulator → Run/Continue” or F5. The code will run until normal termination, error, or hitting a breakpoint (see below).

Alternatively, you can then execute your code one instruction at a time by choosing “Simulator → Single Step” or F10.

You can set a breakpoint at any instruction, where the execution will be paused when control (i.e., the address in the program counter (PC)) reaches that point. To set a breakpoint, hold down ctrl and click on the instruction for which you wish to set a breakpoint. This process can also be used to remove the breakpoint.

4.2 Restarting

Assuming your code is not affected by initial values in registers in the register file (i.e., it begins by loading into the registers the values it needs), you re-run your code by holding down ctrl and clicking on the PC (top left of the Int Regs screen) and selecting “Change Register Contents”. My code was also assigned a starting address of 0x400000 (hex), so to restart, I set PC to 40000, and then run again.

You can instead select to clear all registers in the menu. If you then click on “Run Parameters” (sic), a box appears that states “Address or label to start program” - in which you can enter your desired starting address (it seems to generally pick the right starting location).

4.3 Debugging

Debugging MIPS can be very challenging. However, you can have the simulator step through the code one instruction at a time. Also, you can set a “breakpoint” at an instruction where the simulator will pause execution upon reaching that instruction. To set a breakpoint, place your cursor over the instruction and hold down “ctrl” while clicking on the mouse button. You will see an option to set (or clear) a breakpoint.